ShopEase
Smart E-Commerce Platform

Under the Guidance of
Arugonda Jeevan

Submitted by:
Pavan K

Agenda

- Introduction
- Problem Statement & Proposed Solution
- Objectives
- Technology Stack
- System Architecture
- Usecase Diagram
- Activity Diagram
- Implementation
- Application Demonstartion
- Challenges & Learnings

Introduction

ShopEase is a modern e-commerce platform designed to provide customers with a smooth and reliable online shopping experience. It allows users to browse products, manage their cart, place orders, and track purchases efficiently.

The system is built using Spring Boot microservices, Spring Cloud, and React JS to ensure scalability, fault tolerance, and a responsive user interface. The project focuses on solving key challenges such as payment failures, incorrect inventory updates, and delayed order confirmations by designing a robust and modular architecture

Problem Statement

- Payment gateway failures, causing failed or duplicated transactions
- Incorrect or inconsistent inventory updates leading to stock mismatches
- Delayed order confirmations and shipment notifications
- Result: customer dissatisfaction, refunds, and operational inefficiencies

Proposed Solution

- Microservices architecture (Spring Boot) with Eureka service discovery
- Reliable order flow: inventory reservation + compensating actions on failure
- Idempotent payment handling with webhook verification, retries, and rollback paths
- Centralized error handling and API contracts (SpringDoc OpenAPI)
- Persistence per service using MySQL + Spring Data JPA
- External integrations: payment gateway, shipping API, and email notifications
- Responsive React (Vite) frontend for seamless user experience

Objectives

- To build a scalable and modular e-commerce platform using Spring Boot microservices.
- To provide a smooth shopping experience for users to browse products, manage carts, and place orders.
- To ensure reliable order processing with better error handling and fault tolerance.
- To maintain accurate inventory updates and reduce stock mismatches.
- To improve user experience through a responsive React JS frontend and secure backend services.

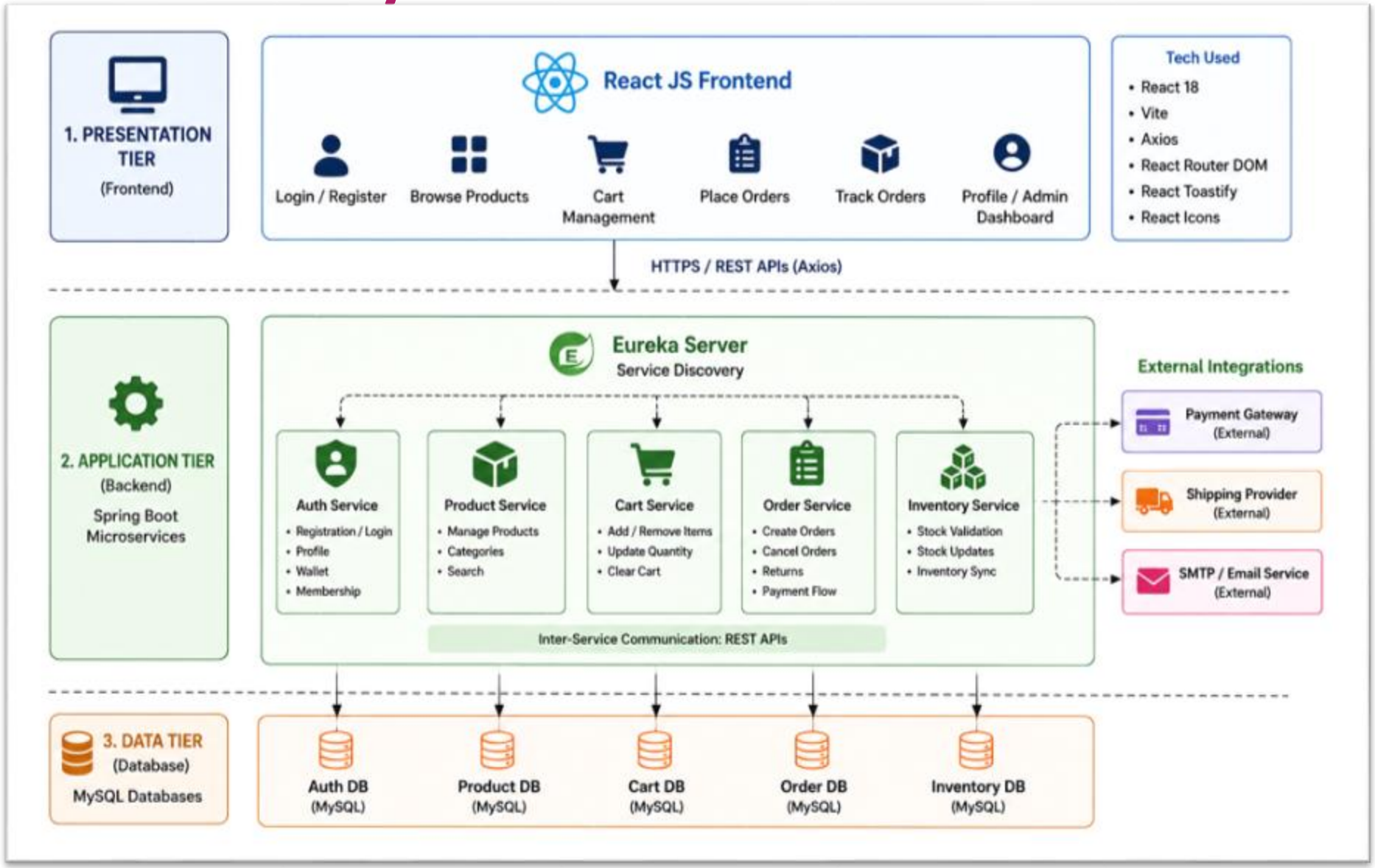
Techonology Stack

Category	Technologies Used
Frontend	React JS, Vite, React Router DOM, Axios, React Icons, React Toastify
Backend	Java 17 + Spring Boot Microservices (Auth, Product, Cart, Order, Inventory)
Service Discovery	Spring Cloud Netflix Eureka Server, Eureka Client
Persistence	MySQL via Spring Data JPA (mysql-connector-j)
API Docs & Testing	SpringDoc OpenAPI (Swagger UI), Spring Boot Starter Test

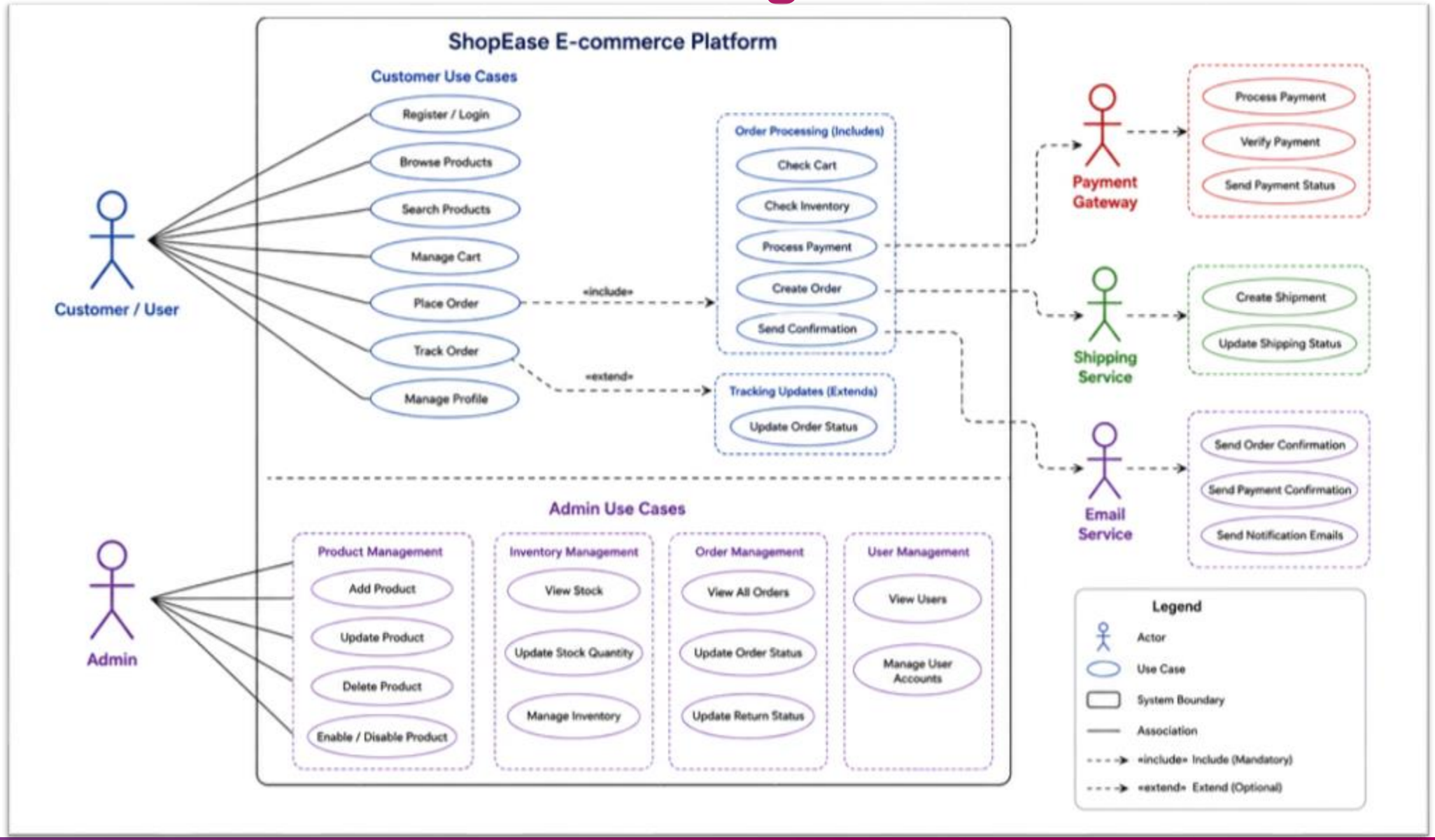
Techonology Stack

Category	Technologies Used
Security & Email	Spring Security (Crypto), Spring Boot Mail
Integrations	Payment Gateway(Razorpay) & Shipping APIs (Endpoints + Webhooks Implemented)
Tooling & Build	Maven (Multi-Module), npm + Vite, Startup Scripts for Local Runs

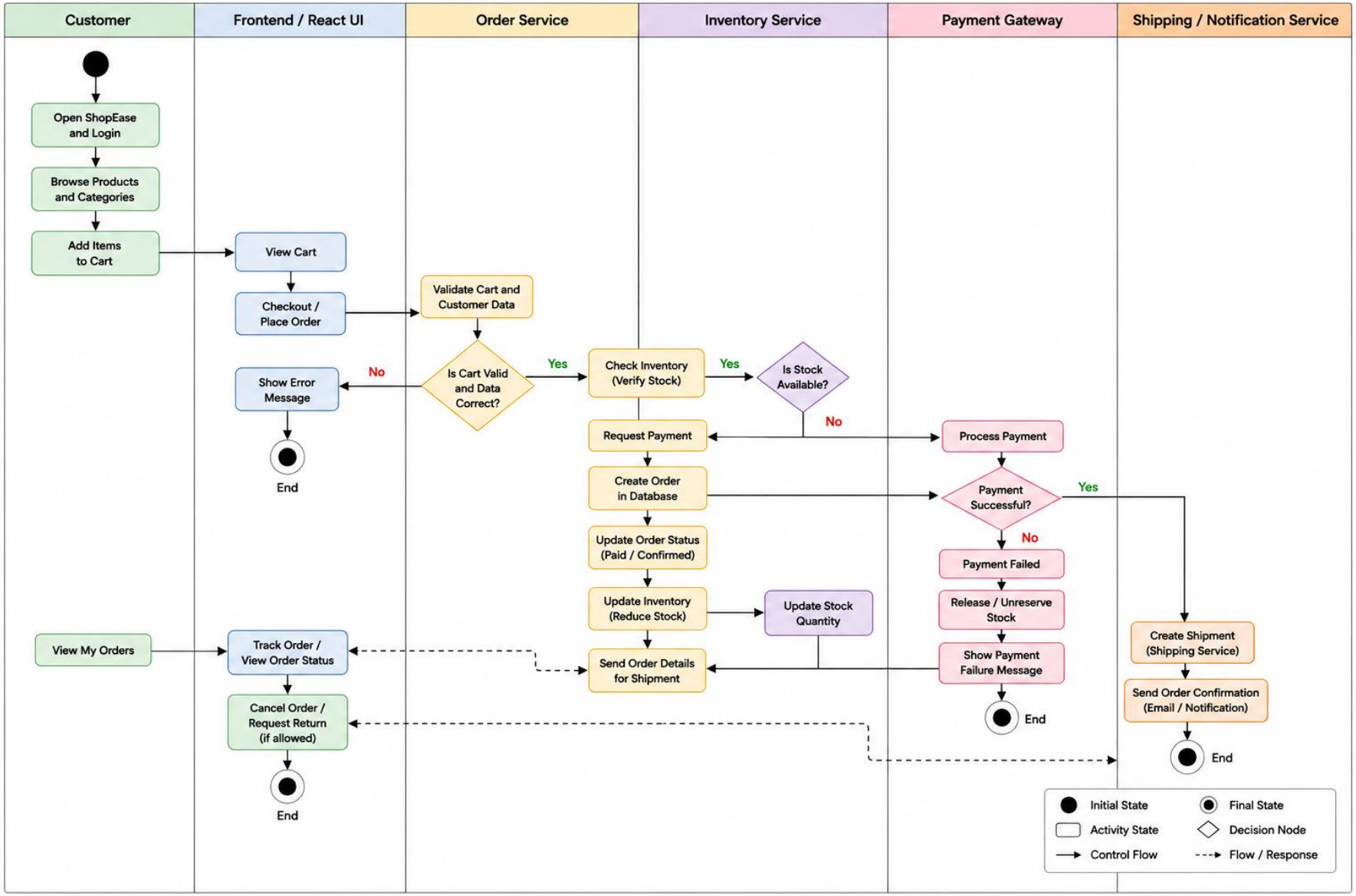
System Architecture



Usecase Diagram



Activity Diagram



Implementation

Layer	Components	Technologies
Frontend Layer (UI / User Experience)	Login & Registration, Home Dashboard, Category Navigation, Product Listing, Category-wise Product View, Cart, Checkout, Order Tracking, Profile, Admin Pages, Wishlist, Chatbot, Promo Modal	React 18, Vite, React Router DOM, Axios, React Icons, React Toastify, CSS
Backend Layer (Business Logic & Data)	Auth Service, Product Service, Cart Service, Order Service, Inventory Service, Eureka Server, REST Controllers, Payment Integration, Shipping Hooks, Email Notifications, Exception Handling, Service-to-Service Communication	Java 17, Spring Boot, Spring Cloud Eureka, Spring Data JPA, Spring Security Crypto, SpringDoc OpenAPI, MySQL, Maven
Database Layer	User Data, Product Data, Cart Records, Order Records, Inventory Stock, Wallet Details, Membership Details, Payment Records, Return Status, Logs & Audit Data	MySQL, JPA/Hibernate

Application Demonstration

Challenges & Learnings

- **Managing microservices communication:** We learned how to connect multiple services using REST APIs and Eureka-based service discovery.
- **Maintaining order and inventory consistency:** Handling stock updates, order placement, and cancellations required careful service coordination.
- **Integrating external services:** Payment and shipping APIs needed proper error handling, retry logic, and verification flows.
- **Frontend-backend integration:** React UI had to match backend DTOs and API responses correctly for smooth user experience.
- **Fault handling and resilience:** We understood the importance of centralized exception handling, validation, and fallback-friendly design.

Thank You For Reviewing our Work